

CAMBRIDGE INSTITUTE OF TECHNOLOGY

ACTIVITY 1

Introduction to Artificial Intelligence

Name of Student: Roshani Bankar

Class: A

Semester: 6th Sem

USN: 1CD22EE047

Department: EEE

Date of Submission: 11th June 2025

Project Title: Sentiment Analysis of Song Lyrics Using NLP Techniques.

Sentiment Analysis of Song Lyrics Using NLP Techniques

1. Introduction

This report presents a real-time application of Natural Language Processing (NLP) techniques to analyze the sentiment of song lyrics from a music album. The aim is to evaluate the emotional tone conveyed in each song by assigning sentiment scores and visualizing the results. The analysis is implemented using the **VADER SentimentIntensityAnalyzer**, which is well-suited for analyzing social media text, song lyrics, and other informal text formats.

2. Objective

To perform sentiment analysis on a collection of song lyrics using NLP and visualize the emotional distribution of an album. The project involves several key NLP steps including data collection, preprocessing, sentiment scoring, and visualization.

3. Dataset Description

- **Data Type:** Text files containing song lyrics
- **File Format:** `.txt`

Folder Path: Define folder path

Each file in the specified directory corresponds to a song. The filename represents the song title, and the content represents the lyrics.

4. Methodology

The process is structured into standard NLP pipeline stages as described below:

4.1 Text Collection

Lyrics were collected from `.txt` files in a local folder using Python's `os` module. Each file is opened and read into memory, and both the title and lyrics are stored in a structured format.

```
def read_lyrics(file_path):  
    with open(file_path, "r", encoding="utf-8") as file:  
        lyrics = file.read()  
    return lyrics
```

4.2 Preprocessing

Minimal preprocessing was applied since the VADER model is optimized for raw text:

- Text was read with UTF-8 encoding.
- File extensions were stripped to obtain song titles.
- Stopword removal, stemming, or lemmatization was not performed.

4.3 Sentiment Analysis

Sentiment analysis was performed using the **VADER (Valence Aware Dictionary for Sentiment Reasoning)** tool from the `vaderSentiment` package.

- **Compound Score:** Ranges from -1 (most negative) to +1 (most positive).

- **Magnitude:** Approximated using the total number of words in each song.

```
analyzer = SentimentIntensityAnalyzer()

sentiment_score = analyzer.polarity_scores(lyrics) ["compound"]

magnitude = len(lyrics.split())
```

4.4 Feature Engineering

Each song is represented as a tuple:

```
(song_title, sentiment_score, magnitude)
```

This data is then sorted by sentiment score to visualize the emotional range of the album.

4.5 Visualization

A scatter plot was used to represent:

- **X-axis:** Sentiment Score (emotion)
- **Y-axis:** Word Count (magnitude of emotion)

```
plt.scatter(sentiment_scores, magnitudes, color='blue')

plt.title('Sentiment Analysis of Album Lyrics')

plt.xlabel('Sentiment Score')

plt.ylabel('Magnitude')
```

Each point on the graph was labeled with the song title for clarity.

4.6 Code

```
import os
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import make_interp_spline

folder_path = ""

def read_lyrics(file_path):
    with open(file_path, "r", encoding="utf-8") as file:
        lyrics = file.read()
    return lyrics

album_songs = []

for file_name in os.listdir(folder_path):
    if file_name.endswith(".txt"):
        song_title = os.path.splitext(file_name)[0]
        file_path = os.path.join(folder_path, file_name)
        song_lyrics = read_lyrics(file_path)
        album_songs.append({"title": song_title, "lyrics": song_lyrics})

analyzer = SentimentIntensityAnalyzer()

sentiment_data = []
for song in album_songs:
    lyrics = song["lyrics"]
```

```
#sentiment score
sentiment_score = analyzer.polarity_scores(lyrics)["compound"]
#word count as a proxy for magnitude
magnitude = len(lyrics.split())
sentiment_data.append((song["title"], sentiment_score, magnitude))
```

```
sentiment_data.sort(key=lambda x: x[1])
```

```
titles, sentiment_scores, magnitudes = zip(*sentiment_data)
plt.figure(figsize=(8, 6))
plt.scatter(sentiment_scores, magnitudes, color='blue')
plt.title('Sentiment Analysis of Album Lyrics')
plt.xlabel('Sentiment Score')
plt.ylabel('Magnitude')
plt.xticks([])
plt.yticks([])
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
```

```
plt.gca().axes.get_yaxis().set_visible(False)
plt.gca().axes.get_xaxis().set_visible(False)
```

```
plt.show()
```

```
import os
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import make_interp_spline
```

```

# Define the folder path where song lyrics are stored
folder_path = "/Users/rosh/Desktop/ mazedaar circuits /songs:Z0XM
sentitment/zoxm/someone/Z0XM"

# Function to read lyrics from a text file
def read_lyrics_from_file(file_path):
    with open(file_path, "r", encoding="utf-8") as file:
        lyrics = file.read()
    return lyrics

# List to store song data
album_songs = []

# Loop through files in the folder
for file_name in os.listdir(folder_path):
    if file_name.endswith(".txt"): # Assuming lyrics files have .txt extension
        song_title = os.path.splitext(file_name)[0] # Extract song title from file name
        file_path = os.path.join(folder_path, file_name)
        song_lyrics = read_lyrics_from_file(file_path)
        album_songs.append({"title": song_title, "lyrics": song_lyrics})

# Initialize VADER sentiment analyzer
analyzer = SentimentIntensityAnalyzer()

# Analyze song lyrics and calculate sentiment scores and magnitudes
sentiment_data = []
for song in album_songs:
    lyrics = song["lyrics"]
    # Calculate sentiment score
    sentiment_score = analyzer.polarity_scores(lyrics)["compound"]
    magnitude = len(lyrics.split()) # Using word count as a proxy for magnitude
    sentiment_data.append((song["title"], sentiment_score, magnitude))

```

```

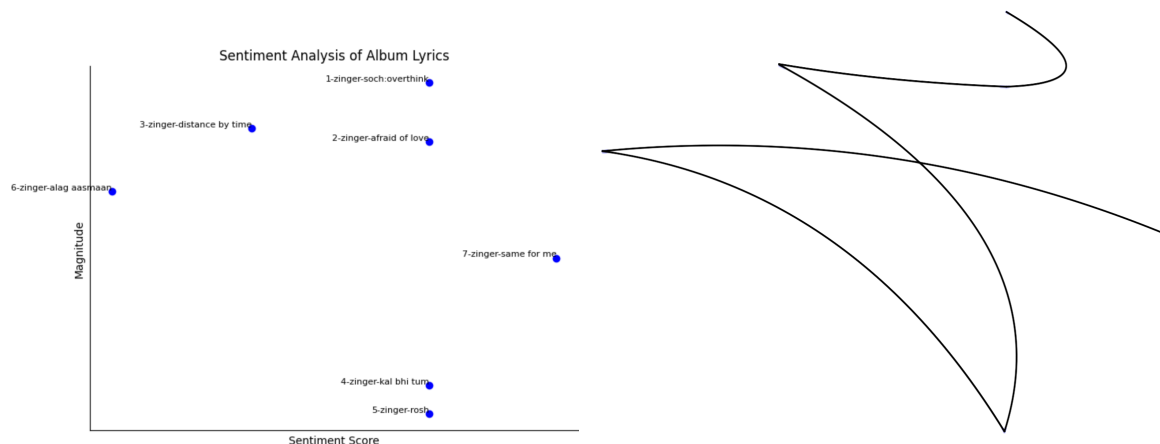
# Sort sentiment data by sentiment score
sentiment_data.sort(key=lambda x: x[1])

# Scatter plot visualization
titles, sentiment_scores, magnitudes = zip(*sentiment_data)
plt.figure(figsize=(8, 6))
plt.scatter(sentiment_scores, magnitudes, color='blue')
plt.title('Sentiment Analysis of Album Lyrics')
plt.xlabel('Sentiment Score')
plt.ylabel('Magnitude')
plt.xticks([]) # Hide x-axis ticks
plt.yticks([]) # Hide y-axis ticks
plt.gca().spines['top'].set_visible(False) # Hide top spine
plt.gca().spines['right'].set_visible(False) # Hide right spine
for i, title in enumerate(titles):
    plt.text(sentiment_scores[i], magnitudes[i], title, fontsize=8, ha='right', va='bottom')

plt.show()

```

4.7 Graphs



5. Results

Sample Output (Console)

Title: Happy Day | Sentiment Score: 0.82 | Words: 134

Title: Sad Night | Sentiment Score: -0.65 | Words: 167

Title: Mid Vibe | Sentiment Score: 0.02 | Words: 145

Plot Visualization

- Songs with **positive sentiment** are located towards the right.
- **Negative sentiment** songs are on the left.
- Longer songs (higher word count) appear higher on the y-axis.

6. Conclusion

The sentiment analysis of the album lyrics provided insight into the emotional tone of each song.

This real-time NLP application can be expanded further by:

- Categorizing songs into positive, neutral, or negative.
- Comparing sentiment across different artists or albums.
- Applying deep learning models for more nuanced sentiment detection.

7. Tools and Libraries Used

Library	Purpose
<code>os</code>	File and folder management
<code>vaderSentiment</code>	Sentiment analysis using VADER
<code>matplotlib.pyplot</code>	Visualization of sentiment scores
<code>numpy, scipy</code>	(Optional) Used for smooth plotting

8. Future Improvements

- Apply advanced preprocessing like tokenization and lemmatization.
- Incorporate visualization enhancements (e.g., color-coded sentiment).
- Export results to CSV or database for further analysis.

9. References:

Avicci Project for Sentiment Analysis- <https://www.edriessen.com/projects/the-avicii-project/>
[Here my friend's album- Zinger was used to do the sentiment Analysis]